

Understanding observability metrics

What is observability metrics

- Observability metrics are numerical measurements that show how a system is performing.
- They help engineers monitor things like:
 - CPU usage
 - Memory usage
 - Request rate
 - Response time
 - Error rate
- Metrics allow teams to detect problems early and understand system behavior over time

Why Metrics Are Important Metrics

help organizations:

- Monitor system health
- Detect outages early
- Improve performance
- Analyze trends over time
- Set alerts for abnormal behaviour

3 Pillars of Observability

1.Metrics:

- Metrics are raw numerical data points collected from hardware, software, and websites.
- metrics are used for monitoring resource usage, performance, and user behaviour.
- metrics tell monitoring and observability teams what is happening in their systems.

core types of metrics:

1.Application metrics:

Application metrics are quantitative measurements that track the health, performance, and user experience of software systems. They identify issues like slow load times, high error rates, and resource bottlenecks (CPU/memory) to ensure system reliability. Key metrics include [response time](#), [throughput](#), [error rates](#), and resource utilization.

2.System metrics:

System metrics are quantitative, time-stamped measurements used to monitor the performance, health, and resource utilization of IT infrastructure, including servers, containers, and cloud instances

3.Business metrics:

Business metrics are quantifiable, data-driven measures used to track, monitor, and assess the success, efficiency, and overall health of specific business processes or objectives. They provide objective evidence of performance, allowing leaders to make informed decisions, identify areas for improvement, and evaluate progress toward strategic goals

2. Logs

Logs are timestamped entries of specific events generated by systems, applications, networks, and infrastructure. They provide event details and context, allowing engineers to understand why issues occur.

Network devices, applications, operating systems, applications, IoT devices, and third-party applications emit different types of logs, including (but not limited to):

- **System logs:** These include events like connection attempts, errors, and configuration changes.
- **Application logs:** They record software changes, CRUD operations, application authentication, and other events to help diagnose issues.
- **Network logs:** Network logs record data from events that take place on a network or device, including network traffic, security events, and user activity.

Logs are recorded in structured and unstructured formats, which represents a storage challenge. They can also be hard to categorize since log data is often siloed in a variety of systems and not automatically correlated.

3.Traces:

- A trace is the complete journey of a single request as it travels through different services in a system.
- Traces allow engineers to monitor and debug applications, discovering bottlenecks.
- Traces tell [DevOps](#) teams **where** issues are occurring in their environments. They're the foundation of proactive monitoring.
- By analyzing traces, engineers can discover which metrics or logs are related to a particular issue, mitigating future issues.

4 Golden signals for SRE teams:

Latency:

- Latency measures the time it takes for data to travel from one point to another.
- Latency will signal underlying performance issues.
- High latency can degrade user experiences by increasing load times, causing application errors, and challenging user expectations.

Traffic

- Traffic metrics track the volume of requests or transactions an application processes.
- They help teams understand user behavior and anticipate scaling needs.

Errors

- Error metrics provide visibility into failed requests or operations.
- Monitoring error rates and identifying patterns can help address recurring issues.

Saturation

- Saturation metrics indicate how close a system is to its capacity limits.
- Monitoring resource utilization ensures that engineers can proactively address bottlenecks before they impact performance.